

Blokk 1

Jinja og templating i dbt

Du har allerede brukt Jinja

Hver `{{ }}` i SQL-filene dine er Jinja:

```
select * from {{ ref('stg_crm__customers') }}  
select * from {{ source('crm', 'crm_customers') }}
```

dbt bruker Jinja til å gjøre SQL **dynamisk** — malen kompileres til vanlig SQL før den sendes til databasen.

Jinja brukes i Python-webrammer, konfigurasjonssystemer og mange andre steder. I dbt er det den mekanismen som gjør makroer, pakker og `ref()` mulig.

To typer Jinja-blokker

SYNTAKS	NAVN	BRUKES TIL
<code>{{ ... }}</code>	Uttrykksblokk	Sett inn en verdi i teksten
<code>{% ... %}</code>	Setningsblokk	Kontrollstrukturer: <code>if</code> , <code>for</code> , <code>set</code>
<code>{# ... #}</code>	Kommentarblokk	Fjernes helt ved kompilering

```
-- Uttrykksblokk – setter inn et tabellnavn
select * from {{ ref('stg_crm__customers') }}

-- Kommentar – vises ikke i kompilert SQL
{# Kunder uten kontrakt holdes i resultatet #}

-- Setningsblokk – setter inn ingenting, men påvirker logikken
{% set status_codes = ['active', 'suspended', 'terminated'] %}
```

Variabler

`{% set %}` definerer en variabel. `{{ }}` setter inn verdien:

```
{% set kolonne = 'contract_status' %}

select
  customer_id,
  {{ kolonne }}
from {{ ref('int_customers__enriched') }}
```

Variabler kan holde tekst, tall, lister og ordbøker:

```
{% set grense = 1000 %}
{% set statuser = ['active', 'suspended', 'terminated'] %}
{% set config = {"materialized": "table"} %}
```

For nå er det nok å vite at `{% set %}` definerer og `{{ }}` bruker.

Betingelser

`{% if %}` og `{% endif %}` omslutter kode som bare genereres under visse betingelser:

```
{% if target.name == 'dev' %}  
    limit 1000  
{% endif %}
```

`target.name` er et innebygd dbt-objekt med informasjon om miljøet du kjører mot.

Med `{% else %}` :

```
{% if target.name == 'prod' %}  
    where status = 'active'  
{% else %}  
    limit 1000  
{% endif %}
```

target-objektet

dbt eksponerer informasjon om kjøremiljøet via `target`:

VARIABEL	EKSEMPEL	BESKRIVELSE
<code>target.name</code>	<code>'dev'</code> , <code>'prod'</code>	Profilnavn i <code>profiles.yml</code>
<code>target.schema</code>	<code>'dbt_dittnavn'</code>	Ditt personlige schema
<code>target.database</code>	<code>'my_project'</code>	BigQuery-prosjekt / Snowflake-database
<code>target.type</code>	<code>'bigquery'</code> , <code>'duckdb'</code>	Databasetype

Nyttig for å skrive logikk som oppfører seg ulikt i dev vs. prod — uten å endre SQL-filene.

Løkker

`{% for %}` itererer over en liste:

```
{% set status_codes = ['active', 'suspended', 'terminated'] %}

select
  customer_id
  {% for s in status_codes %}
    , sum(case when status = '{{ s }}' then 1 else 0 end) as {{ s }}_count
  {% endfor %}
from {{ ref('stg_crm__customers') }}
group by 1
```

`loop.last` er `true` på siste iterasjon — nyttig for å unngå kommaer:

```
{% if not loop.last %},{% endif %}
```


Filtre

Jinja har innebygde **filtre** som bearbeider en verdi med `|`:

```
{{ ' aktiv ' | trim }}           -- 'aktiv' (fjerner mellomrom)
{{ 'aktiv' | upper }}           -- 'AKTIV'
{{ kolonne_liste | join(', ') }} -- 'a, b, c'
```

Du finner filtre i flere makroer, som `generate_schema_name`:

```
{{ custom_schema_name | trim }}
```

Filtre lenkes med `|` og evalueres fra venstre mot høyre.

Kompilert SQL

dbt transformerer Jinja til SQL i to steg:

1. **Jinja-kompilering** — malen evalueres, Jinja byttes ut med vanlig SQL
2. **SQL-kjøring** — den genererte SQL-en sendes til databasen

Se hva som faktisk sendes til databasen:

```
dbt compile --select stg__billing_invoices
```

Kompilert SQL ligger i `target/compiled/`. Svært nyttig ved feilsøking — hvis noe feiler, les den kompilerte SQL-en, ikke malen.

Blokk 2

Makroer

Hva er en makro?

En **makro** er en gjenbrukbar Jinja-funksjon. I stedet for å kopiere den samme SQL-logikken i flere modeller, definerer du den én gang i `macros/` og kaller den der du trenger den.

`billing_invoices` har to YYYYMMDD-kolonner:

```
-- uten makro – samme logikk på begge kolonner
strptime(invoice_date, '%Y%m%d')::date as invoice_date,
strptime(due_date, '%Y%m%d')::date    as due_date,
```

Med en makro:

```
{{ parse_yyyymmdd('invoice_date') }} as invoice_date,
{{ parse_yyyymmdd('due_date') }}    as due_date,
```

Slik trenger du bare å endre logikken ett sted, dersom den skal oppdateres.

Definere en makro

En makrofil i `macros/` bruker `{% macro %}` og `{% endmacro %}`:

```
-- macros/parse_yyyymmdd.sql
{% macro parse_yyyymmdd(column_name) %}
    strftime('{{ column_name }}', '%Y%m%d')::date
{% endmacro %}
```

- `column_name` er argumentet, sendes inn når makroen kalles
- Hoveddelen er vanlig SQL med innslag av Jinja
- Makroen genererer SQL-tekst — den finnes ikke i databasen som en funksjon

Kalle en makro

```
-- models/staging/stg_billing_invoices.sql
with source as (
    select * from {{ source('billing', 'billing_invoices') }}
),
renamed as (
    select
        cast(InvoiceID as varchar)           as invoice_id,
        customer_id,
        {{ parse_yyyymmdd('invoice_date') }} as invoice_date,
        {{ parse_yyyymmdd('due_date') }}     as due_date,
        safe_cast(amount_eur as numeric)     as amount_eur,
        status
    from source
)
select * from renamed
```

dbt kompilerer makroen til `strptime(invoice_date, '%Y%m%d')::date`

Argumenter og standardverdier

Makroer kan ha standardverdier på argumenter med `=`:

```
{% macro parse_yyyymmdd(column_name, format='%Y%m%d') %}  
    strftime('{{ column_name }}', '{{ format }}')::date  
{% endmacro %}
```

Med standardformat (dekker de fleste tilfellene):

```
{{ parse_yyyymmdd('invoice_date') }}
```

Med overstyrt format — om et annet kildesystem bruker et annet mønster:

```
{{ parse_yyyymmdd('created_at', format='%d.%m.%Y') }}
```

Standardverdier gjør makroer fleksible uten at kalleren alltid trenger å spesifisere alt.

Flerdatabasestøtte med `adapter.dispatch`

`strptime()` er DuckDB-syntaks. BigQuery og Snowflake bruker andre funksjoner.

`adapter.dispatch` lar dbt velge riktig implementasjon basert på databasen:

```
{% macro parse_yyyymmdd(column_name) %}  
    {{ return(adapter.dispatch('parse_yyyymmdd')(column_name)) }}  
{% endmacro %}  
  
{% macro default__parse_yyyymmdd(column_name) %}  
    to_date('{{ column_name }}', 'YYYYMMDD')  
{% endmacro %}  
  
{% macro duckdb__parse_yyyymmdd(column_name) %}  
    strptime('{{ column_name }}', '%Y%m%d')::date  
{% endmacro %}  
  
{% macro bigquery__parse_yyyymmdd(column_name) %}  
    parse_date('%Y%m%d', '{{ column_name }}')  
{% endmacro %}
```


adapter.dispatch — navnekonvensjon

dbt leter etter implementasjoner med prefikset `<adapter>__`:

```
{% macro default__makronavn() %}      -- fallback for alle databaser
{% macro duckdb__makronavn() %}      -- brukes kun på DuckDB
{% macro bigquery__makronavn() %}    -- brukes kun på BigQuery
{% macro snowflake__makronavn() %}   -- brukes kun på Snowflake
{% macro postgres__makronavn() %}    -- brukes kun på Postgres
```

SQL-en i makroen `parse_YYYYMMDD('invoice_date')` like overalt. dbt tar seg av å sende riktig dialekt til riktig database.

Eksempel: `sample`

`TABLESAMPLE SYSTEM` lar databasen returnere en tilfeldig andel av datablokker i tabellen uten å lese hele tabellen.

Det er nytting for å begrense kostnader under utvikling i BQ, hvor man betaler for antal bytes scannet (ikke returnert). `LIMIT` begrenser bare hvor mange rader du får som output, ikke hvor mange som scannes.

Ulempen er at resultatet er **ikke-deterministisk**: du får ulike rader hver kjøring. Det gjør det vanskeligere å reproducere feil og kan skjule bugs som bare dukker opp med bestemte dataverdier.

Noen databaser støtter native sampling; andre er enklest å begrense med `LIMIT`. Med `adapter.dispatch` og to parametere — `fraction` og `limit` — håndterer makroen begge:

```
{% macro sample(fraction=10, limit=1000) %}  
  {{ return(adapter.dispatch('sample')(fraction, limit)) }}  
{% endmacro %}
```

```
{% macro default__sample(fraction, limit) %}  
  limit {{ limit }}  
{% endmacro %}
```

```
{% macro bigquery__sample(fraction, limit) %}  
  tablesample system ({{ fraction }} percent)  
{% endmacro %}
```

```
{% macro snowflake__sample(fraction, limit) %}  
  sample ({{ fraction }})
```

`generate_schema_name` — overstyring av dbt-standard

dbt har innebygde makroer som kan overstyres ved å definere en makro med samme navn. `generate_schema_name` styrer hvordan skjemanavn settes:

```
{% macro generate_schema_name(custom_schema_name, node) %}  
    {% set default_schema = target.schema %}  
  
    {% if node.resource_type == 'seed' %}  
        {{ custom_schema_name | trim }}  
    {% elif custom_schema_name is none %}  
        {{ default_schema }}  
    {% elif target.name == 'prod' %}  
        {{ default_schema }}_{{ custom_schema_name | trim }}  
    {% else %}  
        {{ default_schema }}  
    {% endif %}  
{% endmacro %}
```

I dette prosjektet: kun i produksjon får schemas prefiket — i dev samles alt i ett schema.

Makroer som generiske tester

Generiske tester (som `unique` og `not_null`) er egentlig makroer. Du kan skrive dine egne:

```
-- macros/test_is_positive.sql
{% macro test_is_positive(model, column_name) %}
    select *
    from {{ model }}
    where {{ column_name }} <= 0
{% endmacro %}
```

Bruk i YAML på samme måte som innebygde tester:

```
columns:
  - name: amount_eur
  tests:
    - is_positive
```

dbt feiler testen hvis makroen returnerer én eller flere rader.

Når bør du lage en makro?

Makroer løser et problem. De introduserer ikke kompleksitet uten grunn.

Lag en makro når:

- Samme SQL-mønster gjentas i tre (vanlig tommelfingerregel) eller flere modeller
- Logikken er databasespesifikk og du støtter flere databaser
- Du overstyrer dbt-standardlogikk (som `generate_schema_name`)
- Du vil lage gjenbrukbare generiske tester

Ikke lag en makro når:

- Det er engangsbruk
- En CTE eller en view løser problemet like bra
- Den gjør modellkoden vanskeligere å lese
- Makroen du planlegger å lage, gjør for mange ting.

Utangspunktet for design av makroer bør være det samme som for funksjoner i programmering for øvrig: den bør gjøre kun en ting, og ikke ha [sideeffekter](#)

Blokk 3

dbt-pakke-økosystemet

Hva er en dbt-pakke?

En **pakke** er et dbt-prosjekt du henter inn som et bibliotek — makroer, modeller og tester som andre har skrevet og vedlikeholder.

I stedet for å skrive standardverktøy selv, importerer du dem fra pakker som allerede er testet på tvers av mange prosjekter.

```
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.3.0
  - package: dbt-labs/dbt_codegen
    version: 0.13.1
  - package: elementary-data/elementary
    version: 0.15.2
```

```
dbt deps    # laster ned pakker til dbt_packages/
```

dbt Package Hub

Alle offisielle pakker er publisert på hub.getdbt.com.

Noen pakker å kjenne til:

PAKKE	HVA DEN GIR
<code>dbt_utils</code>	Generelle makroer og tester
<code>dbt_codegen</code>	Genererer YAML-boilerplate automatisk
<code>elementary</code>	Datakvalitetsovervåkning og rapportering
<code>dbt_date</code>	Datefunksjoner med tidssonebehandling
<code>audit_helper</code>	Sammenlign to modeller rad for rad
<code>dbt_expectations</code>	Rikt sett med datakvalitetstester

`packages.yml` i dette prosjektet

```
packages:  
- package: dbt-labs/dbt_utils  
  version: 1.3.0  
- package: dbt-labs/dbt_codegen  
  version: 0.13.1  
- package: dbt-labs/dbt_date  
  version: 0.10.2  
- package: dbt-labs/audit_helper  
  version: 0.12.0  
- package: elementary-data/elementary  
  version: 0.15.2
```

```
dbt deps
```

Pakkefilene lastes ned til `dbt_packages/`. Legg denne mappen i `.gitignore` — den regenereres av `dbt deps` og skal ikke versjonskontrolleres.

dbt_utils — oversikt

Den mest brukte pakken i økosystemet. Den gir makroer og tester for de vanligste mønstrene.

Noen sentrale makroer:

MAKRO	HVA DEN GJØR
<code>star()</code>	SELECT alle kolonner fra en modell unntatt et utvalg
<code>generate_series()</code>	Generer en rekke tall eller datoer
<code>pivot()</code>	Roter rader til kolonner
<code>union_relations()</code>	UNION ALL på en liste med modeller

Noen sentrale tester:

TEST	HVA DEN SJEKKER
<code>expression_is_true</code>	Vilkårlig SQL-uttrykk evaluerer til <code>true</code>
<code>unique_combination_of_columns</code>	Kombinasjon av kolonner er unik
<code>not_empty_string</code>	Streng er ikke tom
<code>at_least_one</code>	Minst én rad er ikke null

dbt_utils.expression_is_true

Den mest fleksible testen. Den lar deg sjekke et vilkårlig SQL-uttrykk direkte i YAML:

```
models:
  - name: stg__billing_invoices
    columns:
      - name: amount_eur
        tests:
          - dbt_utils.expression_is_true:
              expression: ">= 0 or status = 'credited'"
      - name: due_date
        tests:
          - dbt_utils.expression_is_true:
              expression: ">= invoice_date"
```

Testen feiler hvis `expression` evaluerer til `false` for én eller flere rader.

Erstatter mange singulære tester.

dbt_utils.unique_combination_of_columns

Sjekker at en **kombinasjon** av kolonner er unik, spesielt nyttig for modeller med en kompositt (sammensatt) primærnøkkel:

```
models:
  - name: int_billing__invoice_settlement
    tests:
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns:
            - invoice_id
            - customer_id
```

Den innebygde **unique**-testen sjekker kun én kolonne om gangen.

Nyttig for faktura-betalings-modeller, kontraktshistorikk og andre modeller med sammensatte nøkler.

dbt_utils.star()

Velg alle kolonner unntatt noen få — uten å skrive ut hele listen manuelt:

```
select
  {{ dbt_utils.star(
    from=ref('int_customers__enriched'),
    except=['_loaded_at', '_row_hash']
  ) }}
from {{ ref('int_customers__enriched') }}
```

Kompileres til en eksplisitt `SELECT kolonne1, kolonne2, ...`-liste.

Nyttig i mart-modeller der du vil eksponere nesten alt fra en intermediate-modell uten å liste opp alle kolonner.

dbt_codegen — generer boilerplate

`dbt_codegen` sparer tid på det kjedelige: å skrive YAML manuelt for eksisterende tabeller.

```
# Generer source-YAML for tabeller som finnes i databasen
dbt run-operation generate_source \
  --args '{"schema_name": "raw", "table_names": ["crm_customers", "crm_contracts"]}'

# Generer modell-YAML med alle kolonner fra en eksisterende modell
dbt run-operation generate_model_yaml \
  --args '{"model_names": ["stg__crm_customers", "int_customers__enriched"]}'
```

Outputen printes til terminalen. Kopier inn i riktig YAML-fil og legg til beskrivelser.

generate_source i praksis

```
# Output fra generate_source:
sources:
- name: raw
  tables:
  - name: crm_customers
    columns:
    - name: custID
      description: ""
    - name: full_name
      description: ""
    - name: email
      description: ""
    - name: address
      description: ""
    - name: preferences
      description: ""
```

Start her, og legg til `description:`, `loaded_at_field` og `freshness` etter behov.

generate_model_yaml i praksis

```
# Output fra generate_model_yaml:  
models:  
  - name: stg__crm_customers  
    description: ""  
    columns:  
      - name: customer_id  
        description: ""  
      - name: customer_name  
        description: ""  
      - name: email  
        description: ""  
      - name: created_date  
        description: ""
```

Alle kolonner fra modellen, inkludert de du nettopp har omdøpt og castet. Legg til tester og beskrivelser.

`generate_model_yaml` krever at modellen allerede er bygget i databasen. Kjør `dbt run` først.

elementary: datakvalitetsovervåkning

elementary overvåker dataene dine over tid — ikke bare om én test passerer akkurat nå, men om noe er i ferd med å endre seg.

Elementary gir:

- **Anomalideteksjon** — varsler hvis radantall, nullandel eller verdidistribusjon endrer seg uventet
- **Testhistorikk** — sporer om tester passerer eller feiler over tid
- **HTML-rapport** — oversikt over hele prosjektets datakvalitet

I motsetning til statiske terskler (`warn_after: 24 timer`) **lærer elementary** hva som er normalt basert på historikk.

elementary — installasjon og oppsett

Legg til i `packages.yml` og `dbt_project.yml`:

```
# dbt_project.yml
models:
  elementary:
    +schema: elementary
    +materialized: incremental
```

```
dbt deps
dbt run --select elementary    # bygger elementarys sporingsmodeller
```

```
pip install 'elementary-data[bigquery]' # eller duckdb / snowflake / postgres
edr report                               # generer HTML-rapport
```

Elementary lagrer testresultater og modellmetadata i egne tabeller i databasen.

elementary — anomalitester i YAML

```
models:
  - name: stg__billing_invoices
    tests:
      - elementary.volume_anomalies:
          timestamp_column: invoice_date
      - elementary.freshness_anomalies:
          timestamp_column: invoice_date
    columns:
      - name: amount_eur
        tests:
          - elementary.column_anomalies:
              column_anomalies:
                - null_count
                - average
                - max
      - name: status
        tests:
          - elementary.column_anomalies:
              column_anomalies:
                - null_count
```

elementary — rapport

`edr report` genererer en lokal HTML-rapport:

```
edr report
```

Rapporten viser:

- Testresultater over tid — grønn/gul/rød per test per kjøring
- Radantall per modell per kjøring
- Skjemaendringer som har skjedd automatisk
- Lineage-visualisering

Nyttig for daglig overvåkning, post-mortem-analyse og som dokumentasjon for stakeholders som vil forstå datakvaliteten uten å lese SQL.

Velge og vedlikeholde pakker

Ikke alle pakker passer alle prosjekter. Noen vurderinger:

SPØRSMÅL	IMPLIKASJON
Er pakken aktivt vedlikeholdt?	Sjekk siste release-dato på GitHub
Støtter den dine databaser?	BigQuery, Snowflake og Postgres har ulike dialekter
Er versjonen festet?	Aldri <code>version: latest</code> i produksjon
Hvem eier <code>dbt deps</code> i CI?	Pakkeversjoner må oppdateres bevisst, ikke automatisk

Pakker er avhengigheter. Oppdater dem som kode, ikke som infrastruktur.

Oppsummering

Jinja:

- `{{ }}` setter inn en verdi, `{% %}` er kontrollstruktur, `{# #}` er kommentar
- `{% set %}`, `{% if %}`, `{% for %}` er de tre du bruker oftest

Makroer:

- Definer én gang i `macros/`, kall overalt
- `adapter.dispatch` gir flerdatabasestøtte med samme kall
- Generiske tester er makroer med signaturen `test_<navn>(model, column_name)`

Pakker:

- Konfigureres i `packages.yml`, installeres med `dbt deps`
- `dbt_utils` — tester og makroer for vanlige mønstre
- `dbt_codegen` — generer YAML-boilerplate fra eksisterende tabeller og modeller
- `elementary` — anomalideteksjon og historisk testovervåking